

ONLINE COMPLAINT REGISTRATION SYSTEM

1. INTRODUCTION

The **Online Complaint Registration System** is a web-based client-server platform designed to automate and simplify public grievance logging workflows. Traditional physical filing methods often lead to structural communication gaps, lost transaction histories, and highly delayed status tracing timelines. This project addresses these limitations by establishing a secure workspace where citizens can file issues instantly and administrators can manage them from a centralized control console.

Project Identification Architecture

- **Project Title:** Production-Grade Online Complaint Registration System
- **Framework Tier:** Full Stack MERN (MongoDB, Express.js, React.js, Node.js) Architecture
- **System Capabilities:** Cryptographic Session Authorization, Dynamic Pipeline Status Transitions, Ingestion Data Sanitization.

2. PROJECT OVERVIEW

Purpose

The core objective of this application is to eliminate administrative delays and optimize accountability. By providing a secure portal accessible over standard web clients, the system streamlines information routing from

the presentation layer to storage clusters, ensuring immediate feedback loops for tracking progress without requiring manual office visits.

Key Features & Functionalities

- **Dual-Interface Role-Based Separation:** Distinct execution dashboards tailored specifically for Citizen and System Administrator personas.
- **Cryptographic Route Safeguards:** Session control managed through JSON Web Token (JWT) signatures paired with server-side decryption middleware.
- **Grievance Intake Sanitization:** Form data fields are checked on the client side and validated on the backend to prevent execution dropouts or database pollution.
- **Real-time State Machine Progress Tracker:** Complainants can trace the resolution status through verified immutable state steps: Pending → In-Progress → Resolved .

3. ARCHITECTURE

Frontend (React.js Architecture)

The presentation space operates as a decoupled Single Page Application (SPA) powered by React.js. It manages client-side routing via a virtual layout structure, isolating authenticated routes behind authorization guards. System states are dispatched asynchronously through non-blocking HTTP networking utilities to interact cleanly with backend endpoints.

Backend (Node.js & Express.js Tier)

The backend application logic runs on a Node.js runtime managed by an Express.js controller framework. It features a modular multi-tier structure split cleanly into routing nodes, request sanitizers, validation checkpoints, and document controller models. Route authorization is enforced by a custom server interceptor that inspects incoming cryptographic header patterns.

Database Schema & Data Model (MongoDB Cluster)

Data persistence is managed via an agile, non-relational MongoDB database. Rather than relying on rigid column constraints, schemas are configured through structured BSON collection frameworks with explicit data constraints. Single and compound indices are implemented across primary lookup parameters to keep query processing speeds fast.

4. SETUP INSTRUCTIONS

Prerequisites

To run, compile, and configure the full-stack system environment locally, ensure the following software tools are installed on your workstation:

- `Node.js` (v16.x or newer runtime engine)
- `npm` (Node Package Manager included with Node.js installation)
- `MongoDB Community Server` (v5.x or newer running on local port 27017, or a remote Atlas URI connection)

Installation & Environment Configurations Step-by-Step

Execute the following commands within your workspace terminal to prepare dependencies and set up system configurations:

```
# 1. Clone your project workspace directory layout
git clone https://github.com/academic-repo/online-complaint-system.git
cd online-complaint-system

# 2. Setup environment variables configuration parameters for the backend server
cat <<EOF > server/.env
PORT=5000
MONGODB_URI=mongodb://localhost:27017/complaint_db
JWT_SECRET=ACADEMIC_PORTFOLIO_SECURE_TOKEN_STRING
EOF

# 3. Install backend dependencies from project root
cd server
npm install

# 4. Install frontend presentation layer dependencies
cd ../client
npm install
```

5. FOLDER STRUCTURE

The project follows a standard decoupled structural pattern, cleanly separating presentation view layouts from backend business logic:

```

online-complaint-system/ └─ client/                                # Frontend Presentation
Architecture (React SPA) └─ public/                                # Static assets and
index files
├─ src/ └─ components/      # Reusable structural
components (Navbar,
ProtectedRoute) └─ pages/    # View screens (Login,
CitizenDashboard,
AdminPanel)
├─ App.js                  # Application root and route path configurations
├─ index.js                # Frontend DOM mount initialization entrypoint
└─ server/                 # Backend API Microservices Tier (Node + Express)
    └─ config/              # Database connectors and collection setups
└─ middleware/              # Security validations and JWT authorization route
guards └─ models/           # MongoDB data validation schemas
    └─ routes/              # Core API request endpoints routing └─
controllers/                # Logic handlers processing database operations
    └─ .env                 # System application variable keys
    └─ app.js               # Express setup orchestration file

```

6. RUNNING THE APPLICATION

To launch the platform locally, the backend microservice tier and the frontend web server must run concurrently on your machine:

Backend Service Engine Launch

```

cd server
npm start
# Server initiates execution on target communication gateway: http://localhost:
5000

```

Frontend Presentation Server Launch

```

cd client
npm start
# Web compiler bundle executes on target access link: http://localhost:3000

```

7. API DOCUMENTATION

The application logic tier exposes structured REST API request paths to safely manage system state transformations:

Method	API Endpoint Route	Access Guard	Payload / JSON Response Target Code
POST	<code>`/api/auth/register`</code>	Public Access	Creates user record. Returns HTTP 201 with registration status token.
POST	<code>`/api/auth/login`</code>	Public Access	Authenticates credentials. Returns HTTP 200 with active user <code>JWT</code> .
POST	<code>`/api/complaints/submit`</code>	Citizen Role	Ingests complaint body fields. Saves data and outputs a tracking ID.
GET	<code>`/api/complaints/track`</code>	Citizen Role	Fetches real-time status history matching the user's ID record.
PATCH	<code>`/api/complaints/status`</code>	Admin Role	Updates status values (Pending → In-Progress → Resolved).

8. AUTHENTICATION

Security validation across the architecture follows an explicit stateless token approach. During login, passwords are encrypted and matched using deep salted comparison hashing. Once verified, the backend generates a signed **JSON Web Token (JWT)** carrying embedded role permissions (`citizen` or `admin`).

This validation token is cached by the client application and attached to downstream outgoing server requests inside the `Authorization: Bearer <TOKEN>` header. The server's validation controller handles incoming payloads, dropping unauthorized attempts with an immediate `HTTP 401 Unauthorized` response code before any application logic executes.

9. USER INTERFACE LAYOUT

The visual framework uses clean components to ensure clear layouts for users throughout their workflow sessions. Key structural interfaces include:

- **Secure Access Interface:** Clean login panels equipped with automated fields that capture input variations and toggle visibility depending on your current role authorization.
- **Citizen Operational Space:** A dynamic workspace presenting a complaint entry form alongside a historical record table. This allows citizens to view clear state changes without manual verification.
- **Administrative Analysis Suite:** A comprehensive control station providing sorting tools to quickly organize high-volume ticket backlogs and update tracking states cleanly.

10. TESTING STRATEGY

System quality control operations were performed using automated test validations coupled with formal component verification matrices:

Test Key	Target Module	Execution Conditions & Verification Inputs	Result
TC-01	Route Middleware Guard	Send unauthenticated data payloads to <code>/api/complaints/submit</code> with missing verification headers.	PASSED
TC-02	Form Field Sanitizer	Attempt data record processing with empty form fields or empty text payloads.	PASSED
TC-03	Admin State Controls	Submit state transformation queries using an authorized administrative login account.	PASSED

11. KNOWN ISSUES & OPERATIONAL BOUNDS

During deployment testing, minor connection limits were cataloged. During Sprint 3 development checks, cross-origin resource sharing (CORS) interface changes caused temporary dropouts when sending frontend payloads to the decoupled server tier. This issue was resolved by configuring explicit communication route setups inside the backend network logic. The current system configuration successfully processes automated validation actions without application faults.

12. FUTURE ENHANCEMENTS

Planned expansions for upcoming production releases include integrating automated alert pipelines using realtime notification brokers to message users during tracking state changes. Additionally, incorporating machinelearning classifiers to read intake text and auto-assign correct category tags would further streamline the administrative routing process.